

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>

#define MAX_COMMANDS 16
#define MAX_ARGS 32
#define BUF_SIZE 1024

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Parse command arguments for an individual stage
int parse_cmd_args(char *cmd_str, char **args) {
    int count = 0;
    char *token = strtok(cmd_str, " \t");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    return count;
}

// Execute large multi-stage pipeline and monitor resource metrics
void execute_pipeline(char *line) {
    char *commands[MAX_COMMANDS];
    int num_cmds = 0;

    // Split pipeline by '|' delimiter
    char *token = strtok(line, "|");
    while (token != NULL && num_cmds < MAX_COMMANDS) {
        commands[num_cmds++] = trim_space(token);
        token = strtok(NULL, "|");
    }
    if (num_cmds == 0) return;

    struct timespec start_time, end_time;
    clock_gettime(CLOCK_MONOTONIC, &start_time);

```

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Tex

```

    int count = 0;
    char *token = strtok(cmd_str, " \t");
    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
    args[count] = NULL;
    return count;
}

// Execute large multi-stage pipeline and monitor resource metrics
void execute_pipeline(char *line) {
    char *commands[MAX_COMMANDS];
    int num_cmds = 0;

    // Split pipeline by '|' delimiter
    char *token = strtok(line, "|");
    while (token != NULL && num_cmds < MAX_COMMANDS) {
        commands[num_cmds++] = trim_space(token);
        token = strtok(NULL, "|");
    }
    if (num_cmds == 0) return;

    struct timespec start_time, end_time;
    clock_gettime(CLOCK_MONOTONIC, &start_time);

    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("pipe creation failed");
            return;
        }
    }

    pid_t pids[MAX_COMMANDS];
    for (int i = 0; i < num_cmds; i++) {
        pids[i] = fork();
        if (pids[i] < 0) {
            perror("fork failed during pipeline execution");
            return;
        }

        if (pids[i] == 0) {
            // If not first stage, redirect stdin from previous pipe read end
            if (i != 0) {
                if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) < 0) {
                    perror("dup2 stdin failed");
                    exit(EXIT_FAILURE);
                }
            }
        }
    }

```

^G Get Help	^O WriteOut	^R Read File	^Y Prev Pg	^K Cut Text
^X Exit	^J Justify	^W Where is	^V Next Pg	^U UnCut Tex

```

int pipefds[2 * (num_cmds - 1)];
for (int i = 0; i < num_cmds - 1; i++) {
    if (pipe(pipefds + i * 2) < 0) {
        perror("pipe creation failed");
        return;
    }
}

pid_t pids[MAX_COMMANDS];
for (int i = 0; i < num_cmds; i++) {
    pids[i] = fork();
    if (pids[i] < 0) {
        perror("fork failed during pipeline execution");
        return;
    }

    if (pids[i] == 0) {
        // If not first stage, redirect stdin from previous pipe read end
        if (i != 0) {
            if (dup2(pipefds[(i - 1) * 2], STDIN_FILENO) < 0) {
                perror("dup2 stdin failed");
                exit(EXIT_FAILURE);
            }
        }

        // If not last stage, redirect stdout to current pipe write end
        if (i != num_cmds - 1) {
            if (dup2(pipefds[i * 2 + 1], STDOUT_FILENO) < 0) {
                perror("dup2 stdout failed");
                exit(EXIT_FAILURE);
            }
        }

        // Close all pipe descriptors in child process
        for (int j = 0; j < 2 * (num_cmds - 1); j++) {
            close(pipefds[j]);
        }

        char *args[MAX_ARGS];
        if (parse_cmd_args(commands[i], args) == 0) {
            exit(EXIT_SUCCESS);
        }

        execvp(args[0], args);
        fprintf(stderr, "Pipeline Execution Error: Command '%s' failed: %s\n", args[0], strerror(errno));
        exit(EXIT_FAILURE);
    }
}

// Parent closes all pipe file descriptors
for (int i = 0; i < 2 * (num_cmds - 1); i++) {

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: pipeline_perf_shell.c

```

}

// Wait for all child processes in the pipeline to finish
int failure_detected = 0;
for (int i = 0; i < num_cmds; i++) {
    int status;
    waitpid(pids[i], &status, 0);
    if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        failure_detected = 1;
    }
}

clock_gettime(CLOCK_MONOTONIC, &end_time);
double elapsed = (end_time.tv_sec - start_time.tv_sec) +
    (end_time.tv_nsec - start_time.tv_nsec) / 1e9;

// Monitor resource consumption metrics via getrusage
struct rusage usage;
getrusage(RUSAGE_CHILDREN, &usage);

// Display performance metrics
fprintf(stderr, "\n--- PIPELINE PERFORMANCE REPORT ---\n");
fprintf(stderr, "Stages Executed : %d\n", num_cmds);
fprintf(stderr, "Pipeline Wall Time : %.4f seconds\n", elapsed);
fprintf(stderr, "User CPU Time Spent : %ld.%06ld s\n", usage.ru_utime.tv_sec, usage.ru_utime.tv_usec);
fprintf(stderr, "System CPU Time : %ld.%06ld s\n", usage.ru_stime.tv_sec, usage.ru_stime.tv_usec);
fprintf(stderr, "Max Resident Set : %ld KB\n", usage.ru_maxrss);
fprintf(stderr, "Pipeline Status : %s\n", failure_detected ? "FAILED (Non-zero exit)" : "SUCCESS");
fprintf(stderr, "-----\n\n");
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("PerfPipelineShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_pipeline(input_buf);
    }

    return 0;
}

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

```
zsh: command not found: valgrind
[medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall pipe.c -o pipe
[medikondarakesh@Medikondas-MacBook-Air ~ % ./pipe
PerfPipelineShell > valgrind --leak-check=full --track-fds=yes ./pipe ..
```

--- PIPELINE PERFORMANCE REPORT ---

```
Stages Executed : 1
Pipeline Wall Time : 0.0020 seconds
User CPU Time Spent: 0.000118 s
System CPU Time : 0.001078 s
Max Resident Set : 1245184 KB
Pipeline Status : FAILED
-----
```

PerfPipelineShell > # Direct execution under Valgrind

valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes ./pipePipeline Execution Error: Command '#' failed: No such file or directory

--- PIPELINE PERFORMANCE REPORT ---

```
Stages Executed : 1
Pipeline Wall Time : 0.0041 seconds
User CPU Time Spent: 0.000180 s
System CPU Time : 0.001698 s
Max Resident Set : 1245184 KB
Pipeline Status : FAILED
-----
```

PerfPipelineShell >

Pipeline Execution Error: Command 'valgrind' failed: No such file or directory

--- PIPELINE PERFORMANCE REPORT ---

```
Stages Executed : 1
Pipeline Wall Time : 0.0022 seconds
User CPU Time Spent: 0.000130 s
System CPU Time : 0.001096 s
Max Resident Set : 1245184 KB
Pipeline Status : FAILED
-----
```

PerfPipelineShell > cat /etc/passwd | grep -v nologin | wc -l

```
System CPU Time : 0.001007 s
Max Resident Set : 1376256 KB
Pipeline Status : SUCCESS
-----
```

PerfPipelineShell > ls -l /usr/bin | grep python | sort | head -n 5
-rwxr-xr-x 78 root wheel 200560 24 Sep 12:40 python3

--- PIPELINE PERFORMANCE REPORT ---

```
Stages Executed : 4
Pipeline Wall Time : 0.0593 seconds
User CPU Time Spent: 0.007867 s
System CPU Time : 0.034351 s
Max Resident Set : 2097152 KB
Pipeline Status : SUCCESS
-----
```

PerfPipelineShell > cat non_existent_file.txt | grep test | wc -l
cat: non_existent_file.txt: No such file or directory
0

--- PIPELINE PERFORMANCE REPORT ---

```
Stages Executed : 3
Pipeline Wall Time : 0.0070 seconds
User CPU Time Spent: 0.003469 s
System CPU Time : 0.008078 s
Max Resident Set : 2097152 KB
Pipeline Status : FAILED
-----
```

PerfPipelineShell > exit

medikondarakesh@Medikondas-MacBook-Air ~ %